Essential Python Terminologies and Concepts

LS100 — Module 00B · Python Fundamentals

Souvik Mandal, Ph.D.[11] Project Leader & Instructor, LS100, FAS, Harvard University

Souvik Mandal, Ph.D., Linkedin ID: souvik-mandal-phd

Project Leader & Instructor, Computational Behavioral Sciences, LS100, FAS, Harvard University

Below is a list of key Python terminologies every beginner should understand, along with brief explanations:

- **Interpreter:** The Python **interpreter** is the program that executes your Python code. Python is an *interpreted* language, meaning you don't compile your code into machine language before running it – instead, the interpreter reads and runs your code line by line[1]. When you installed Python, you obtained an interpreter (accessible via the `python`/`python3` command or IDLE). If the interpreter encounters an error in your code, it will stop and display an error message (traceback) indicating what went wrong.
- **Program:** A Python **program** is simply a script or application written in Python – a set of instructions for the computer to execute. You can think of a program like a recipe: it contains steps (commands) that the computer follows to perform a task. Even a short Python script that prints a message is a program. Applications like web browsers or games are also programs (often composed of many files and modules). In general, *a program is a sequence of instructions* that tells the computer what to do[1].
- **Syntax and Indentation: Syntax** refers to the set of rules that define how Python code must be written. Python's syntax is known for being clean and readable. One unique aspect is that **indentation (leading whitespace on lines)** is meaningful in Python. Instead of using braces `{}` or keywords like `endif` to designate code blocks, Python uses indentation levels. For example, the lines inside a function or inside an `if` statement must be indented consistently. If the indentation is wrong or inconsistent, the interpreter will raise an `IndentationError`. Indentation thus determines the structure of code blocks[1]. This requirement makes Python code visually neat and helps enforce good formatting habits. Other syntax rules include using a colon `:` to start an indented block (e.g., after `def` for a function, or after `if` conditions), and the use of line breaks to end statements (one statement per line, unless you use a backslash or other means to continue).
- **Comments:** In Python, a **comment** is text in the code meant for humans, not for execution. Comments are created by prefixing text with the `#` symbol. For example: `# This is a comment.` The Python interpreter ignores anything on a line after a `#`, so comments do not affect the program's behavior. They are used

Published Jul 01, 2026

to explain code or leave notes. *Comments are code lines that will not be executed*[2]. Python also has a way to write multi-line comments or docstrings using triple quotes (`""" ... """`), commonly used to document functions or modules.

- **Variable:** A **variable** is a named storage location that holds a value. You can think of it as a container or a label for data. In Python, you create a variable by assigning a value to a name, for example: `count = 10`. Here, `count` is a variable name that refers to the value `10`. Variables make it possible to store and manipulate data in programs. Internally, a variable name points to an object representing the data in memory. *Basically, a variable is a place in computer memory with an assigned name, used to store data of different types (text, numbers, etc.)*[1]. Unlike some languages, you don't declare a type for a variable in Python – the type is inferred from whatever value you assign to it, and you can later change the value (and type) of the variable if needed.
- **Data Types (Numbers, Strings, Booleans, etc.):** A **data type** defines what kind of value a piece of data is, which determines what you can do with it. Common Python data types include:
- **Integer (`int`):** whole numbers (e.g. `5`, `-42`).
- **Floating-point number (`float`):** numbers with a decimal point or in exponential form (e.g. `3.14`, `2.0`, `5e6` for 5×10^6).
- **String (`str`):** sequence of characters used to represent text (e.g. `"Hello"` or `'Python'`). Strings are enclosed in quotes. Python strings can be manipulated with many methods since they are a **sequence** type (you can think of them as a sequence of characters). For instance, you can find its length, slice it, or split it into parts. (Examples: `"Hello"[1]` gives `"e"`, and `"Hello".split("e")` gives `["H", "llo"]`.)[3].
- **Boolean (`bool`):** logical values `True` or `False`. Booleans often result from comparisons (like `5 < 10` yields `True`) and are used in control flow (if/while conditions). In Python, `True` and `False` are reserved keywords representing these two truth values.
- **NoneType:** a special type with a single value `None`, used to represent "no value" or the absence of data.

Python is *dynamically typed*, meaning you don't need to specify the type of a variable and a variable's type can change when you assign a new value. You can always check the type of a value by using the `type()` function (e.g. `type(42)` returns `<class 'int'>`).

- **Operator:** An **operator** is a symbol (or sometimes a keyword) that tells the interpreter to perform a specific operation on one or more operands (values or variables). For example, `+` is an arithmetic operator that adds two numbers or concatenates strings. Python supports many types of operators:
- **Arithmetic operators** (`+`, `-`, `*`, `/`, `//` for floor division, `%` for modulus, `**` for exponentiation) to perform mathematical calculations[2].
- **Comparison (Relational) operators** (`==` equals, `!=` not equals, `>`, `<`, `>=`, `<=`) to compare two values, yielding a boolean result[2].

- **Logical (Boolean) operators** (`and`, `or`, `not`) to combine boolean conditions (`and` & `or`) or negate them (`not`)[2].
- **Assignment operators** (like `=` to assign, `+=`, `-=`, etc. to modify and assign) for setting variable values[2].
- **Identity operators** (`is`, `is not`) to check if two references point to the same object in memory.
- **Membership operators** (`in`, `not in`) to test if a value is a member of a sequence or collection (e.g. `x in my_list`)[2].

Using operators, you can form expressions like `a * b + 5` or `if (x > 0 and x < 10): ...`. The operators follow standard precedence (e.g. multiplication comes before addition, unless parentheses are used to group).

- **Conditional (If) Statement: If statements** are a fundamental control structure that let your program make decisions. An `if` statement executes a block of code only if a certain condition is true. In Python:
- ```
  if condition:
      # code block
  elif other_condition:
      # code block
  else:
      # code block
  ```

You start with `if` and a condition (an expression that evaluates to True or False). If the condition is True, the indented block under the `if` runs; if it's False, the program can optionally check other conditions with `elif` (else-if) or execute a default block under `else`. For example:

```
age = 20
if age < 18:
    print("Minor")
else:
    print("Adult")
```

Here, "Adult" will be printed because the condition `age < 18` is False, so it goes to the `else`. Conditions often use comparison operators (==, >, <, etc.) and logical operators (`and`, `or`) to combine multiple criteria[1]. Python uses a colon `:` after the condition and indentation to mark the block of code that belongs to the `if`. Proper indentation is crucial, as mentioned earlier, to ensure the `if` body is recognized[1]. If statements allow programs to branch and perform different actions based on input or state.

- **Loop (For / While): Loops** allow you to execute a block of code repeatedly. Python has two main loop types:
- **For loop:** used to iterate over elements of a sequence (like a list, tuple, set, string, or range of numbers). In Python, a `for` loop looks like `for item in collection: ...`. It will assign each element of the collection to `item` and execute the indented block for each element[1]. For example:
```

- `fruits = ["apple", "banana", "cherry"]`
  `for fruit in fruits:`
  `    print(fruit)`

This will print each fruit name in the list. A `for` loop is ideal when you know in advance how many elements (or iterations) you need to go through.

- **While loop:** used to repeat a block as long as a condition remains true. It looks like `while condition: ...`. The loop will keep running, checking the condition before each iteration, and it will stop once the condition becomes false[1]. For example:
- `count = 5`
  `while count > 0:`
  `    print(count)`
  `    count -= 1`

This will print 5, 4, 3, 2, 1 each on a new line, then stop when count is 0 (since the condition `count > 0` is no longer true). You must ensure that the condition will eventually become false, otherwise a while loop can run forever.

Both loop types use indentation to define the loop body. You can use the keyword `break` to exit a loop early, or `continue` to skip to the next iteration. In general, *for loops* are common when iterating over known collections or ranges, and *while loops* are useful when the repetition should continue until a condition changes (which might depend on user input or some calculation). Loops are essential for tasks like iterating through data, repeating an operation multiple times, or searching through collections[1].

- **Function:** A **function** is a reusable block of code that performs a specific task. You can define your own functions using the `def` keyword, giving the function a name and optional parameters. For example:
- `def greet(name):`
  `    print("Hello,", name)`

defines a function `greet` that takes one parameter `name`. Using `greet("Alice")` would print "Hello, Alice". Functions allow you to organize code into logical pieces and avoid repetition – you write the code once and call it whenever needed. *A function is a reusable block of code that performs a certain action (or actions). Functions are used to accomplish tasks, especially those that need to be done repeatedly, with different inputs*[1]. Python also has many **built-in functions** (like `len()`, `range()`, etc.), and you can import functions from modules. When a function needs to produce a result, it can `return` a value, which then can be used by the caller. If a function doesn't explicitly return anything, it returns `None` by default.

- **print() function:** `print()` is one of Python's most basic and frequently used built-in functions. It outputs messages to the screen (or more technically, to the standard output). For example, `print("Welcome to Python")` will display that text. You can print variables and other data types as well. `print()` is very handy for showing results or for debugging by inspecting variable values. *Every Python developer learns to use* `print()` *early, as it allows us to display the result of code or*

messages for the user[1]. By default, `print` adds a newline at the end of what it prints, so each call prints on a new line (this can be changed with an optional parameter).

- **input() function:** `input()` is the counterpart to print – it allows you to **receive input from the user**. When you call `input()`, the program will pause and wait for the user to type something and press Enter. The text the user enters is then returned as a string. You can optionally pass a string to `input()` to display a prompt message (e.g. `name = input("Enter your name: ")`). For example:
- `age_text = input("How old are you? ")`

If the user types "21" and hits enter, then `age_text` will be the string `"21"`. You might then convert this to an integer with `age = int(age_text)`. *The* `input()` *function always returns the user input as a string (even if they typed digits)[1].* Using `input()` is one way to make programs interactive and dynamic.

- **Object:** In Python (an object-oriented language), an **object** refers to an instance of a data type that encapsulates both data and functionality. Practically everything in Python is an object – integers, strings, lists, functions, even modules are objects. Each object has a type (or class) that defines what it is and what it can do. For example, the number `42` is an object of type `int`, and it has certain behaviors (like it can be added or subtracted). A list like `[1,2,3]` is an object of type `list` and has methods like `.append()`. If you define a class (blueprint) for something, the actual realized entities are objects. *In short, an object is a concrete instance of some class or data type, bundling together data and operations*[3]. For a real-world analogy, if you have a class `Car` (concept), then a specific car (like your dad's red Toyota) is an object (instance) of that class.
- **Class:** A **class** is like a blueprint or template for creating objects. It defines a new data type by specifying what attributes (data) and methods (functions) its objects will have. You can define a class in Python using the `class` keyword:
- ```
  class Car:
      def __init__(self, brand, color):
          self.brand = brand
          self.color = color
      def drive(self, miles):
          print(f"Driving {miles} miles")
  ```

Here `Car` is a class with attributes `brand` and `color`, and a method `drive`. From this class, you can create objects (instances) like `my_car = Car("Toyota", "red")`. *A class can be thought of as a blueprint for objects – it defines the structure and behavior (via attributes and methods) that the objects of the class will have*[3]. Python has many built-in classes (for example, `list`, `dict`, `str` are classes), and you can also create your own to model more complex data. One advantage of using classes is that you can create multiple objects from the same blueprint without repeating code, and you can leverage concepts like inheritance (one class inheriting attributes/methods from another). For beginners, it's enough to grasp that classes = blueprints, objects = actual instances.
  ```

- **Attribute:** An **attribute** is a piece of data stored in an object (or class). If we think in terms of Python code, attributes are the **variables/properties that belong to an object or class**. There are two types:
- *Instance attribute:* a piece of data specific to an object (instance). In the `Car` class example, `brand` and `color` are instance attributes; each `Car` object has its own values for these.
- *Class attribute:* a variable that is defined at the class level (not in any method) and is shared by all instances of the class.

Additionally, **methods** (see below) are sometimes called "behavioral attributes" or "procedural attributes" of a class. *Data attributes define what an object needs to have (state), while methods define how to interact with the object (behavior)[3].* For example, if you have an object `person` of class `Person` with attributes `name` and `age`, those are data attributes. If `Person` has a method `speak()`, that method is also considered an attribute of the class (a function attribute). In Python, you access attributes with the dot notation: e.g. `person.name` or `my_car.color`. Trying to access an attribute that doesn't exist will result in an `AttributeError`. Understanding attributes is important when dealing with objects and classes, as it helps organize data and functionality together.

- **Instance:** An **instance** is another name for an object in the context of a class. We say an object is an instance of a particular class. For example, using the `Car` class above, `my_car = Car("Toyota", "red")` creates an instance of `Car`. You can create multiple instances (objects) from one class, each with their own attribute values. *An instance of a class is an individual object that possesses the structure and behavior defined by that class[3].* In conversation, "instance" and "object" are often used interchangeably. The word instance emphasizes the relationship to its class (like "this is an instance of class Car"). Checking an object's type with `type(obj)` will show you its class, and `isinstance(obj, ClassName)` returns True if the object is an instance of that class (or a subclass of it).
- **Method:** A **method** is a function that is defined inside a class and is meant to be called on objects of that class. Methods typically operate on the object's data. For example, in a list object `my_list`, `my_list.append(5)` calls the `append` method of the list, which adds a new element. Methods are described as the behavior of an object. The only difference between a method and a regular function is that a method is tied to an object (the first parameter of a method is usually `self`, referring to the instance). *Methods are similar to functions, but they belong to a particular class and are used to interact with class instances[3].* For instance, `str.upper()` is a method that returns an uppercase version of a string. In our `Car` class example, `drive()` is a method; you would call `my_car.drive(100)`, which would perhaps reduce fuel or print a message. Understanding methods is important because much of Python's functionality comes from calling methods on objects (e.g. list methods to manipulate lists, dict methods to handle dictionaries, etc.).

- **List:** A **list** is a built-in data structure in Python that holds an ordered collection of items. Lists are very flexible – they can contain elements of any type, and you can add, remove, or change items. Lists are created using square brackets, e.g. `numbers = [1, 2, 3, 4]`. The items in a list have a defined order (index starting from 0 for the first item) and the list can be modified (it's *mutable*). You can access elements by index (`numbers[0]` gives `1`), slice lists (`numbers[1:3]` gives `[2,3]`), and use methods like `append()`, `remove()`, `sort()`, etc. *In summary, a list is a data type that can store a sequence of values, accessible by an index (position)[1].* Lists are one of the most commonly used data structures for storing collections of items.
- **Tuple:** A **tuple** is another built-in sequence type, quite similar to a list, but with one key difference: tuples are **immutable**. Once a tuple is created, its contents cannot be changed (you can't add, remove, or modify items in place). Tuples are defined using parentheses, for example `coordinates = (10.0, 20.0)` or `my_tuple = ("a", "b", "c")`. They support indexing and slicing like lists (`coordinates[0]` is `10.0`). Because they are immutable, tuples are useful for representing fixed collections of items (such as a pair of coordinates, or a database record that shouldn't change). Tuples can also be used as keys in dictionaries (since immutability makes them hashable), whereas lists cannot. *In essence, a tuple is a sequence of objects, very much like a list, but defined and fixed upon creation[3].* If you need an ordered collection that you won't need to modify, a tuple is a good choice.
- **Dictionary:** A **dictionary** (dict) is a built-in Python data structure that stores a collection of key–value pairs. It's also sometimes called a hash map or associative array in other languages. You create a dictionary with curly braces `{}`, for example:
- `student = {"name": "Alice", "age": 25}`

Here `"name"` and `"age"` are keys, and `"Alice"` and `25` are their respective values. You retrieve or set values by key, e.g. `student["name"]` gives `"Alice"`. Dictionaries are *unordered* collections (as of Python 3.7+, insertion order is preserved, but conceptually you don't rely on order) and are mutable (you can add, remove, or change entries). Keys in a dict must be unique and immutable (numbers, strings, tuples, etc. can be keys; lists cannot). *A Python dictionary stores an unordered collection of data values, where each element has a key and a value[1].* They are very powerful for representing structured data and performing fast lookups by key (like a word-to-definition mapping, or a username-to-user info mapping). For instance, you could have `prices = {"apple": 0.99, "banana": 0.50}` to map items to their prices.

- **Set:** A **set** is a built-in data type that represents an **unordered collection of unique elements**. Sets are like mathematical sets – they don't allow duplicates and the order of elements is not preserved. You create a set using curly braces with comma-separated values (similar to dict literal but just values, no colons), e.g. `unique_numbers = {1, 2, 5}`. You can also use the `set()` constructor to create a set from another iterable (like a list). Key features of sets:

- Fast membership testing: you can check if an item is in a set with `in` very efficiently.
- Because they only allow unique items, sets are useful for removing duplicates from a collection.
- They support operations like union, intersection, difference (via methods or operators like `|`, `&`, `-`).

In Python, a set is an unordered collection of distinct (unique) immutable objects[3]. For example, `{1,2,3,3}` will end up as `{1,2,3}` because duplicates are removed[3]. Sets themselves are mutable (you can add or remove elements), but because they are unordered, you cannot access elements by index. There is also a special type called a **frozenset** which is an immutable set. Beginners commonly use sets when they need to gather unique items or perform mathematical set operations.

- **String:** We touched on strings earlier as a data type, but to elaborate: a **string** in Python is a sequence of characters used to represent text. Strings are created by enclosing characters in quotes, either single `'...'` or double `"..."` quotes. They are **immutable** (you cannot change a string in place; operations that appear to modify a string actually create a new string). Python provides many helpful operations for strings:
- You can **concatenate** strings with `+` (e.g. `"Hello, " + name`).
- **Length:** use `len(my_string)` to get the number of characters.
- **Indexing and slicing:** you can access characters by index (`s[0]`) or substrings via slicing (`s[start:end]`)[3]. Indices start at 0, and negative indices count from the end (`s[-1]` is the last character)[3].
- **Methods:** There are many built-in methods, e.g. `s.lower()`, `s.upper()`, `s.split(" ")`, `s.strip()`, `s.find("abc")`, etc., to transform or search within the string. For example, `"John Doe".split(" ")` returns `["John", "Doe"]`[3].

Strings are fundamental for input/output, representing messages, file paths, data read from files, etc. Even though they are immutable, operations are designed to be efficient and convenient.

- **Library/Package:** In Python, a **package** (or library or module – the terms are related, see below) is a collection of pre-written code that you can include in your projects to add functionality. The Python Standard Library is the collection of packages/modules included with Python by default (for tasks like math, file I/O, data structures, etc.). Additionally, the Python community has tens of thousands of third-party packages available via the Python Package Index (PyPI). A **package** in the filesystem sense is a directory containing one or more Python modules (usually identified by an `__init__.py` file). However, when we say *"Python package"* colloquially, we often mean any installable library or tool. Examples include `numpy` (for numerical computing), `pandas` (data analysis), `requests` (HTTP requests), etc. *Python offers a rich selection of packages; any developer can create a package and publish it on PyPI for others to install[3].*

Packages are used to **re-use code** – instead of writing something from scratch, you can install a package that provides the functionality you need.

- **Module:** A **module** is a single Python file (with a `.py` extension) that can be imported, or a component of a larger package. When you import a module, you're essentially loading that file and making its functions/classes available in your code. For example, Python's standard library has a module called `math` (contained in `math.py` in the library), and you can do `import math` to use `math.sqrt(16)` etc. If you have a file `utils.py` with some helper functions, `utils` is a module you wrote. *In terms of relationship: a package may contain multiple modules (files), whereas a module is a single file. Often, you will only need to import a specific module from a package rather than the whole package at once[3].* For instance, a large package might have submodules and you could do `from package import submodule`. But if you're not creating your own packages yet, you can think of modules simply as "libraries" or files you import.
- **pip (Package Installer): pip** is the standard package installer for Python. It's the tool you use to **install and manage third-party libraries** from the Python Package Index (and other repositories). For example, if you want to install the requests library, you would run `pip install requests` in your command prompt. Pip will download the package and its dependencies and install them into your Python environment. *In other words, pip is the package installer for Python – you can use pip to install packages from the Python Package Index (PyPI) and other indexes[4].* Pip typically comes bundled with Python installers. To use it, open a terminal and type `pip install <package_name>`. You can also use it to upgrade packages (`pip install --upgrade <name>`) or uninstall (`pip uninstall <name>`). Knowing how to use pip is essential for leveraging Python's vast ecosystem of external libraries. In environments like Anaconda, you might use `conda install` for packages available through conda, but pip can still be used there as well.

References

[1] LearnPython.com. "Python Terminology for Beginners." https://learnpython.com/blog/python-terms-for-beginners/

[2] W3Schools. "Python Glossary." https://www.w3schools.com/python/python_ref_glossary.asp

[3] LearnPython.com. "Python Terminology for Beginners, Part 2." https://learnpython.com/blog/python-terms-for-beginners-2/

[4] Python Packaging Authority. "pip · PyPI." https://pypi.org/project/pip/